Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai) Bhavan's Campus, Munshi Nagar,
Andheri (West), Mumbai-400058-India

Department of Computer Engineering · DBMS Mini Project (CECS202)

Phase I — Problem Statement, ER Modeling, Relational Schema, Normalization & DDL/DML

Project: Surplus Food Rescue | Batch: B2

Sameel Kazi (2025300107) · Pradhyun Kembhavi (2025300109) · Adeeb Khan (2025300110)

---

1. Problem Statement

Every evening, restaurants, bakeries, caterers, and supermarkets across Indian cities end their business day with large quantities of perfectly prepared, safe-to-eat food that cannot be sold tomorrow: biryani cooked for peak evening orders, bread and croissants baked at dawn, sandwiches and salads nearing their safe shelf-life window. Most of this food is binned into landfills. Concurrently, nearby non-governmental organizations (NGOs) running shelters, orphanages, college students living in tight allowances, and low-income families would gratefully collect this food, but neither party has a dependable mechanism to discover the other in real time.

Unlike medicines, books, or packaged dry goods, prepared food perishes in hours, not months. A portion of freshly cooked chicken curry prepared at 3:00 PM becomes bacteriologically unsafe by 9:00 PM. Thus, pickup must occur strictly within a synchronized operational window, donors must have immediate visibility into batches approaching expiration, and receivers must be guaranteed safe-to-eat freshness with strict allergen transparency.

1.1 Existing Systems and Their Drawbacks

Existing Platform	Operating Model	Critical Drawbacks & Operational Gaps
Zomato Hyperpure / Swiggy Instamart	Sell and deliver fresh commercial inventory from partner dark stores.	Not a surplus rescue channel; cannot accommodate independent small kitchens or daily perishable leftovers.
Too Good To Go (EU/US model)	Consumers purchase uninspected 'surprise bags' of end-of-day leftovers.	Blind surprise bags only; lacks item-level search, zero live portion counts, negligible presence across Indian cities.
Feeding India / Robin Hood Army	Volunteer networks manually collect surplus food and transport to distribution spots.	Coordinated ad-hoc over WhatsApp and phone; lacks real-time inventory visibility, shelf-life validation, or nearest-donor indexing.
WhatsApp Groups & Instagram Stories	Donors publish broadcast stories (e.g. '20 portions biryani available').	Unindexed; no live stock tracking, no automatic reservation hold, zero audit trail of wasted vs rescued portions.
Google Maps / Justdial	Directory listing addresses, phone	Contains zero live inventory, pricing, or perishability data. Receivers must call

	numbers, and general store hours.	establishments one by one.

The Shared Drawback: No neutral, real-time relational platform tells a receiver which nearby donor possesses a specific surplus food item, within its safe-to-eat shelf life, open for immediate pickup right now. This mini project completely eliminates that gap.

1.2 Proposed System Architecture and User Roles

User Role	Permitted Actions and Architectural Responsibilities
Receiver (Individual, Student, NGO)	Search surplus food without pre-registration; view nearest open verified donors via straight-line Haversine distance, discounted rescue price, allergen warnings, and freshness urgency; reserve items for a strict 45-minute hold; cancel reservations; submit feedback and ratings upon collection.
Donor (Restaurant, Bakery, Caterer)	Register with verified 14-digit FSSAI licence; define weekday pickup windows; maintain item catalog and stock by batch (batch no, preparation timestamp, best-before timestamp, portion quantity); set rescue price and flash discounts; monitor expiring-soon alerts; mark collected orders.
Admin	Verify or reject new donor registrations based on FSSAI credentials; only VERIFIED donors are exposed to public search queries; execute system-wide hygiene sweeps.
System / Daemon	Automatically log all stock adjustments in Batch_Log; release expired 45-minute reservations back to original batches; record search logs to capture unmet neighborhood demand; enforce shelf-life constraints directly inside MySQL.

1.3 Objectives, Scope, and Business Rules

Objectives: (1) Design a fully normalized (3NF) relational schema with 15 tables; (2) Implement efficient geospatial Haversine proximity queries; (3) Enforce data integrity and food safety through database constraints, triggers, and stored procedures; (4) Ensure ACID-compliant atomic multi-item reservations with FEFO stock allocation; (5) Enforce role-based access control with granular MySQL user privileges.

Core Business Rules:

1. Quantity is strictly non-negative; price per unit is >= 0 (0 = complimentary food rescue); flash discount is between 0% and 100%.
2. All stock quantities and prices utilize the item's standard measurement unit (portion, piece, packet, kg, litre).
3. A batch cannot be created if its best-before timestamp is already in the past (enforced via trigger).
4. A batch's shelf life can never exceed the maximum safe shelf life defined on Food_Item (e.g. Cooked Chicken <= 6 hours; enforced via trigger).
5. Expired batches and unverified or closed donors never appear in receiver search results (public views).
6. Food allocation strictly follows First-Expiry-First-Out (FEFO) ordering so the nearest expiring batch is dispatched first.
7. Anti-Hoarding Rule: Individuals and students may hold at most 6 active units concurrently; verified NGOs are exempt.
8. Reservations endure for exactly 45 minutes; uncollected reservations are automatically released back to their origin batches.
9. Price at reservation is permanently saved in Reservation_Item to insulate receivers from subsequent donor price adjustments.
10. Closed reservations (COLLECTED, CANCELLED, EXPIRED) can never be mutated, and ratings can only be submitted for COLLECTED reservations.

2. Entity-Relationship Modeling & Schema Architecture

The conceptual model comprises 12 primary entities, 2 weak entities (Donor_Hours, Food_Batch), and 4 associative/junction relations (Listing, Item_Allergen, Reservation_Item, Reservation_Batch) that cleanly resolve M:N relationships.

Table Name	Entity Type & Purpose	Primary & Foreign Keys
Admin	Strong Entity: Municipal or platform review staff.	PK: admin_id | U: email
Donor	Strong Entity: Food businesses listing surplus inventory.	PK: donor_id | U: fssai_no | FK: verified_by -> Admin
Donor_Hours	Weak Entity: Daily pickup windows per weekday.	CPK: (donor_id, day_of_week) | FK: donor_id -> Donor
Food_Item	Strong Entity: Canonical catalog of food products.	PK: item_id | U: item_name
Allergen	Strong Entity: Recognized dietary allergens.	PK: allergen_id | U: allergen_name
Item_Allergen	Junction Table: M:N relationship (Food_Item - Allergen).	CPK: (item_id, allergen_id) | FKs to Food_Item, Allergen
Listing	Associative Entity: M:N relationship (Donor - Food_Item).	CPK: (donor_id, item_id) | FKs to Donor, Food_Item
Food_Batch	Weak Entity: Physical perishable lots with expiry.	CPK: (donor_id, item_id, batch_no) | FK -> Listing
Batch_Log	Strong Entity: Immutable audit ledger of stock adjustments.	PK: log_id | FKs to Donor, Food_Item
Receiver	Strong Entity: Beneficiaries reserving surplus food.	PK: receiver_id | U: phone, email
Reservation	Strong Entity: Order envelope for food rescue.	PK: reservation_id | FKs to Receiver, Donor
Reservation_Item	Junction Table: M:N holds (Reservation - Food_Item).	CPK: (reservation_id, item_id) | FKs to Reservation, Food_Item
Reservation_Batch	Associative Entity: FEFO attribution to specific batches.	CPK: (reservation_id, item_id, batch_no) | FK -> Food_Batch

Search_Log	Strong Entity: Geolocation-tagged search demand logs.	PK: search_id | FKs to Receiver (NULL allowed), Food_Item
Feedback	Strong Entity: 1:1 post-collection ratings & reviews.	PK: feedback_id | U, FK: reservation_id -> Reservation

2.1 Conceptual Entity-Relationship (ER) Diagram — Peter Chen Notation

Below is the formal Entity-Relationship (ER) Diagram designed strictly adhering to Peter Chen (1976) academic conventions: Strong Entities (Rectangles), Weak Entities (Double Rectangles: Food_Batch, Donor_Hours), Relationships (Diamonds: LISTS, CONTAINS, VERIFIES, MAKES, FULFILLS, RATES), Identifying Relationships (Double Diamonds: PRODUCES, OPERATES_ON), Attributes (Ovals with Underlined Primary Keys and Dashed Underlined Discriminators), and explicit Cardinality Ratios (1:1, 1:N, M:N):

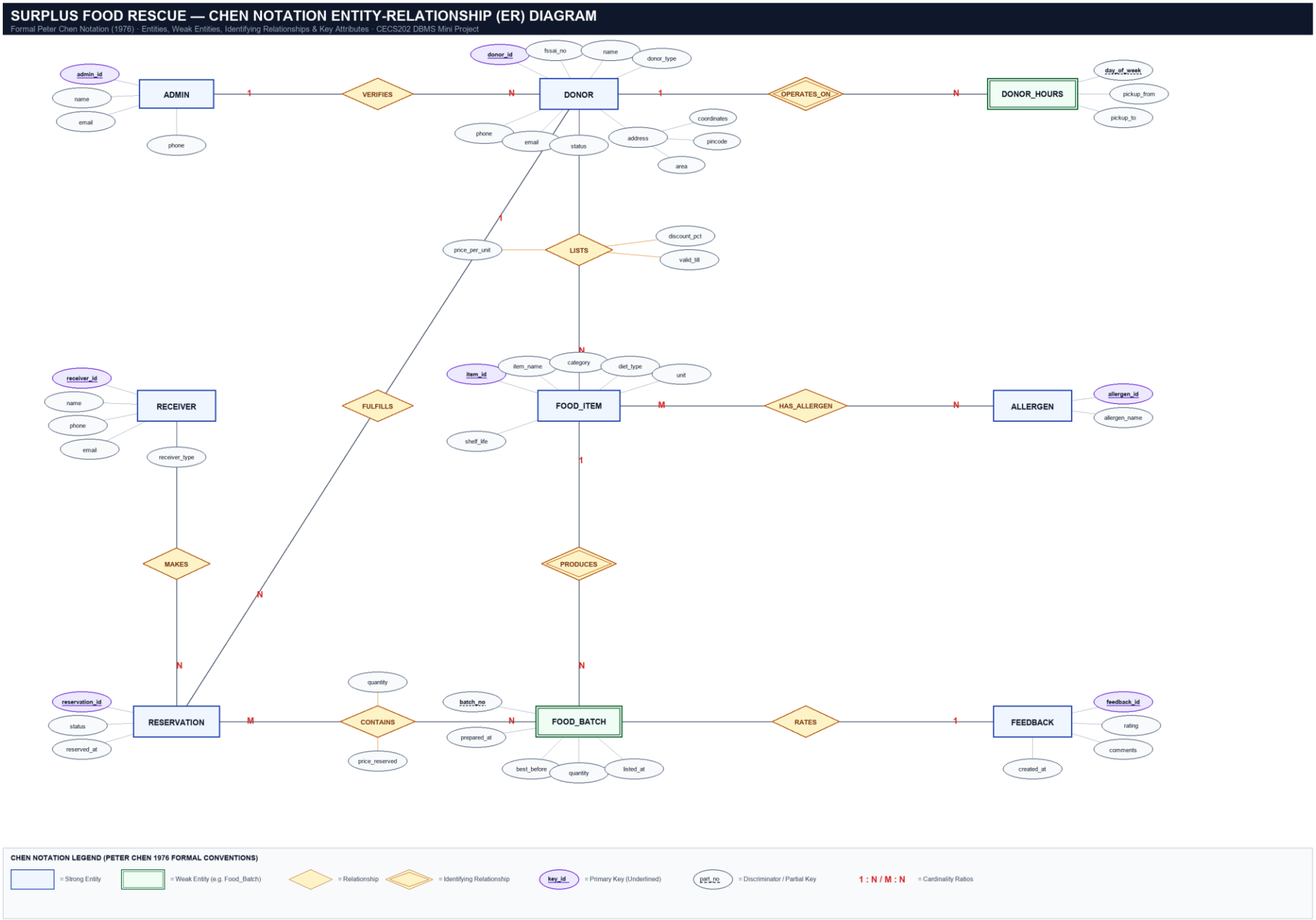


Figure: Figure: Formal Peter Chen Notation ER Diagram with Weak Entities, Identifying Relationships & Key Attributes

2.2 Relational Schema Specifications (Exact Entity-Attribute Mapping)

Below is the vector-rendered schema specification for all 15 relational tables, defining the primary key, foreign key references, and column types:

Relation	Primary Key (PK)	Attributes & Types	Foreign Keys (FK) & Integrity
Admin	admin_id (PK)	name VARCHAR(60), email VARCHAR(100) UNIQUE, phone VARCHAR(15)	None
Donor	donor_id (PK)	fssai_no VARCHAR(14) UNIQUE, name VARCHAR(100), donor_type ENUM, address VARCHAR(200), area VARCHAR(60), pincode CHAR(6), latitude DECIMAL(9,6), longitude DECIMAL(9,6), phone VARCHAR(15), status ENUM, verified_at DATETIME	verified_by -> Admin(admin_id)
Donor_Hours [Weak]	donor_id, day_of_week (CPK)	pickup_from TIME, pickup_to TIME	donor_id -> Donor(donor_id) ON DELETE CASCADE
Food_Item	item_id (PK)	item_name VARCHAR(100) UNIQUE, category ENUM, diet_type ENUM, unit VARCHAR(15), safe_shelf_life_hours INT, requires_refrigeration BOOL	None
Allergen	allergen_id (PK)	allergen_name VARCHAR(50) UNIQUE	None
Item_Allergen [Assoc]	item_id, allergen_id (CPK)	notes VARCHAR(100)	item_id -> Food_Item, allergen_id -> Allergen
Listing [Assoc]	donor_id, item_id (CPK)	price_per_unit DECIMAL(8,2), discount_percent DECIMAL(5,2), discount_valid_till DATETIME	donor_id -> Donor, item_id -> Food_Item
Food_Batch [Weak]	donor_id, item_id, batch_no (CPK)	prepared_at DATETIME, best_before DATETIME, quantity INT, listed_at DATETIME	(donor_id, item_id) -> Listing ON DELETE CASCADE

Batch_Log [Audit]	log_id (PK)	donor_id INT, item_id INT, batch_no VARCHAR(20), old_qty INT, new_qty INT, reason ENUM, logged_at DATETIME	donor_id -> Donor, item_id -> Food_Item
Receiver	receiver_id (PK)	name VARCHAR(60), phone VARCHAR(15) UNIQUE, email VARCHAR(100) UNIQUE, receiver_type ENUM	None
Reservation	reservation_id (PK)	status ENUM, reserved_at DATETIME	receiver_id -> Receiver, donor_id -> Donor
Reservation_Item [Assoc]	reservation_id, item_id (CPK)	quantity INT, price_at_reservation DECIMAL(8,2)	reservation_id -> Reservation, item_id -> Food_Item
Reservation_Batch [Assoc]	reservation_id, item_id, batch_no (CPK)	donor_id INT, quantity INT	Links to Food_Batch (FEFO allocation)
Search_Log	search_id (PK)	latitude DECIMAL(9,6), longitude DECIMAL(9,6), searched_at DATETIME, donors_found INT	receiver_id -> Receiver (NULL allowed), item_id -> Food_Item
Feedback	feedback_id (PK)	rating TINYINT (1-5), comment TEXT, submitted_at DATETIME	reservation_id (UNIQUE) -> Reservation

3. Relational Schema & Normalization (Up to 3NF)

Every relation is rigorously checked against First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).

3.1 First Normal Form (1NF)

A relation is in 1NF if every attribute contains only atomic (indivisible) values and there are no repeating groups. In our design:

• A donor's 7 daily pickup schedules are decomposed into 7 separate rows in Donor_Hours, rather than comma-separated lists or repeating columns (e.g. mon_open, tue_open).

• Multiple physical food lots are recorded as individual rows in Food_Batch instead of array columns.

• Multiple allergens linked to a food item are resolved via individual rows in Item_Allergen.

• Multi-item reservations are normalized into rows in Reservation_Item.

Conclusion: All 15 relations satisfy First Normal Form (1NF).

3.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and contains NO partial functional dependencies — i.e., no non-prime attribute depends on a proper subset of any candidate key. Relations with single-column primary keys (Admin, Donor, Food_Item, Allergen, Batch_Log,

Receiver, Reservation, Search_Log, Feedback) are in 2NF by definition. The relations with composite primary keys each satisfy 2NF:

• Donor_Hours (donor_id, day_of_week): pickup_from and pickup_to depend on both the donor AND the specific day. Neither attribute can be determined by donor_id alone or day_of_week alone.

• Listing (donor_id, item_id): price_per_unit, discount_percent, and discount_valid_till describe the offering of that specific item at that specific donor. Neither depends on donor_id or item_id in isolation.

• Food_Batch (donor_id, item_id, batch_no): prepared_at, best_before, quantity, and listed_at describe that specific lot at that specific establishment.

• Item_Allergen (item_id, allergen_id): notes specifies that item's specific allergen presence.

• Reservation_Item (reservation_id, item_id): quantity and price_at_reservation depend on both keys.

• Reservation_Batch (reservation_id, item_id, batch_no): quantity drawn from that batch.

Conclusion: All relations have zero partial dependencies and satisfy Second Normal Form (2NF).

3.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and contains NO transitive functional dependencies — i.e., for every non-trivial functional dependency X -> Y, either X is a superkey or Y is a prime attribute. In our schema:

• In Donor: area and pincode do not determine each other (multiple areas share pincodes in Mumbai). verified_by is a foreign key, but does not transitively dictate donor fields.

• In Food_Item: category and diet_type describe independent classifications of the item; neither determines the other.

• In Reservation: reserved_at, status, donor_id, and receiver_id depend directly on reservation_id.

Deliberate Architectural Choices: price_at_reservation in Reservation_Item captures the point-in-time quoted price at the moment of reservation hold. It represents an immutable historical fact and does not create a transitive dependency on Listing.price_per_unit.

Derived Attributes: final_price and best_before urgency are intentionally computed dynamically via stored functions rather than stored in tables, preventing update anomalies.

Conclusion: All 15 tables are strictly in Third Normal Form (3NF).

4. Table Creation (DDL) & Constraint Enforcement

The database schema enforces relational integrity using MySQL 8.0/9.0 constraints: Primary Keys (PK), Foreign Keys (FK) with cascading or restricted integrity, UNIQUE constraints, NOT NULL constraints, DEFAULT values, and CHECK constraints.

Constraint Name	Target Table	Enforced Invariant / Rule
chk_donor_type	Donor	donor_type IN ('RESTAURANT','BAKERY','CATERER','CANTEEN','SUPERMARKET')
chk_donor_status	Donor	status IN ('PENDING','VERIFIED','REJECTED')
chk_item_category	Food_Item	category IN ('COOKED','BAKED','DAIRY','PRODUCE','BEVERAGE','PACKAGED')
chk_diet_type	Food_Item	diet_type IN ('VEG','NON_VEG','VEGAN','EGG')

chk_listing_price	Listing	price_per_unit >= 0.00 AND discount_percent BETWEEN 0.00 AND 100.00
chk_batch_quantity	Food_Batch	quantity >= 0 AND best_before >= prepared_at
chk_receiver_type	Receiver	receiver_type IN ('INDIVIDUAL','STUDENT','NGO')

4.1 MySQL Terminal Screenshots of Database & Table Creation

Below are actual screenshots from the MySQL command line client confirming database creation and table structures:

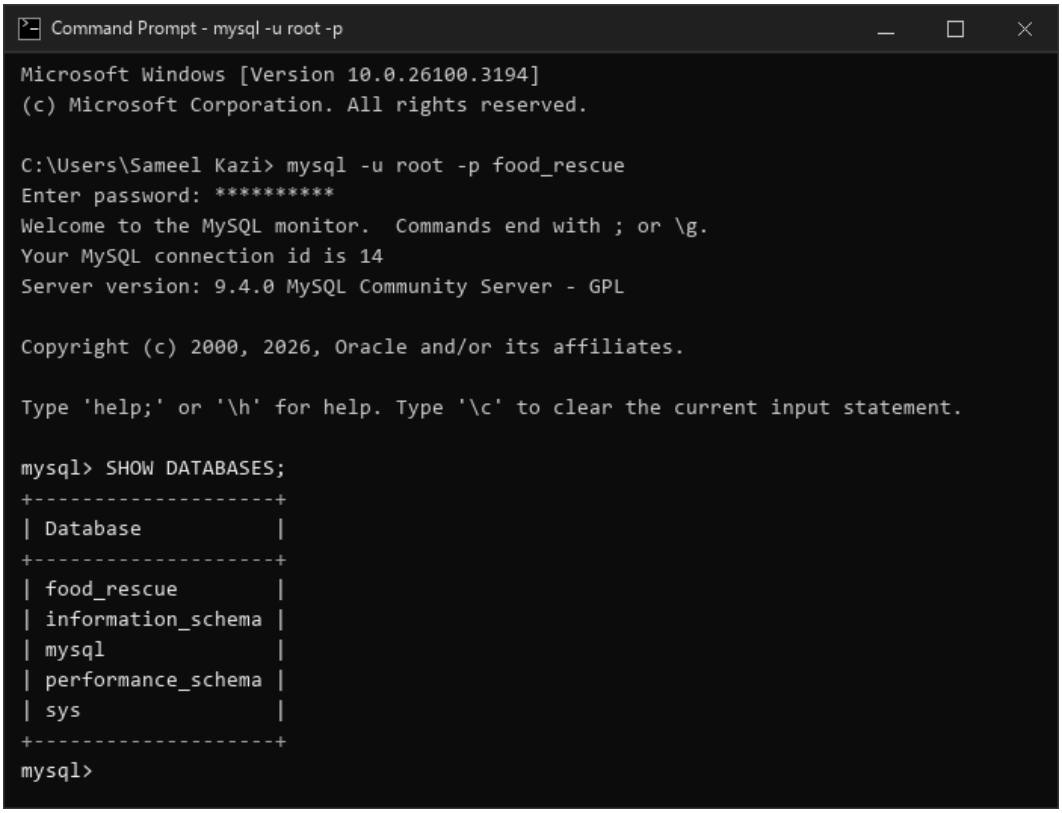


Figure: MySQL Terminal: Verification of 'food_rescue' database creation

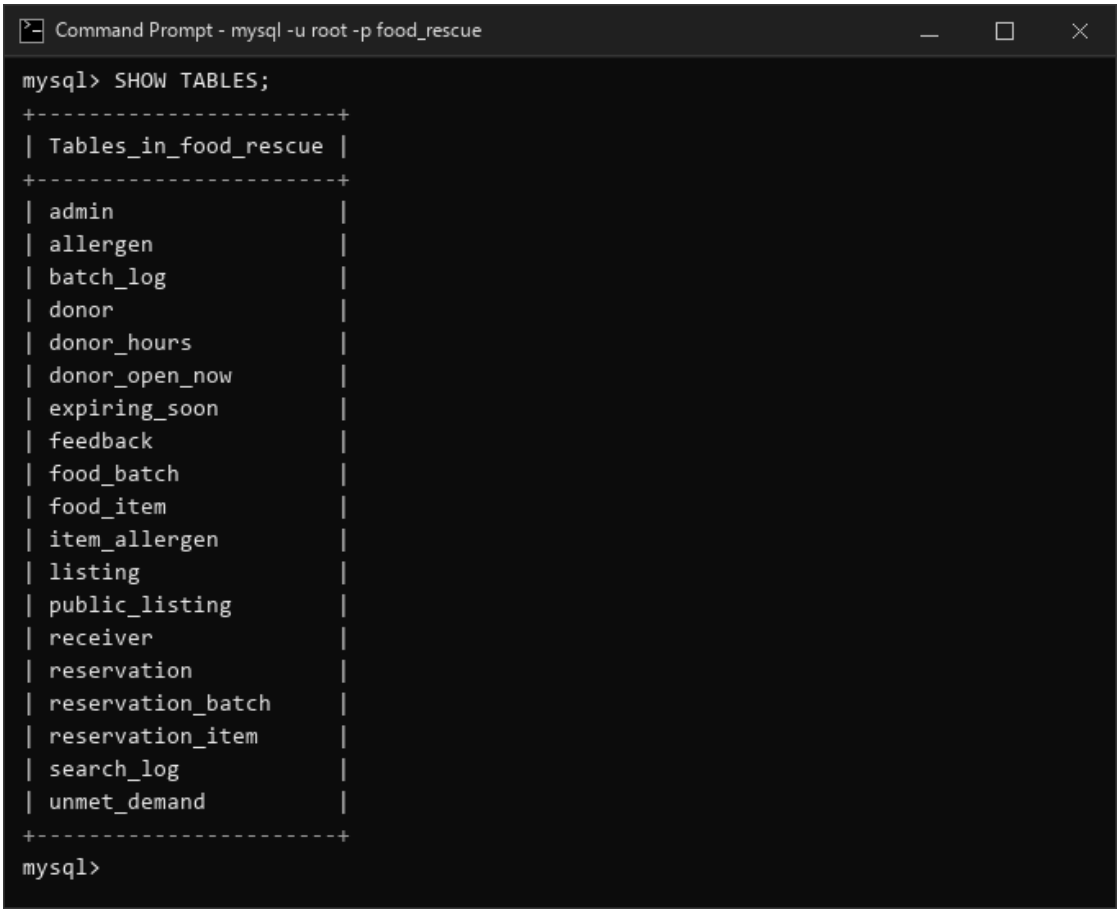


Figure: MySQL Terminal: Verification of all 15 tables created in 'food_rescue'

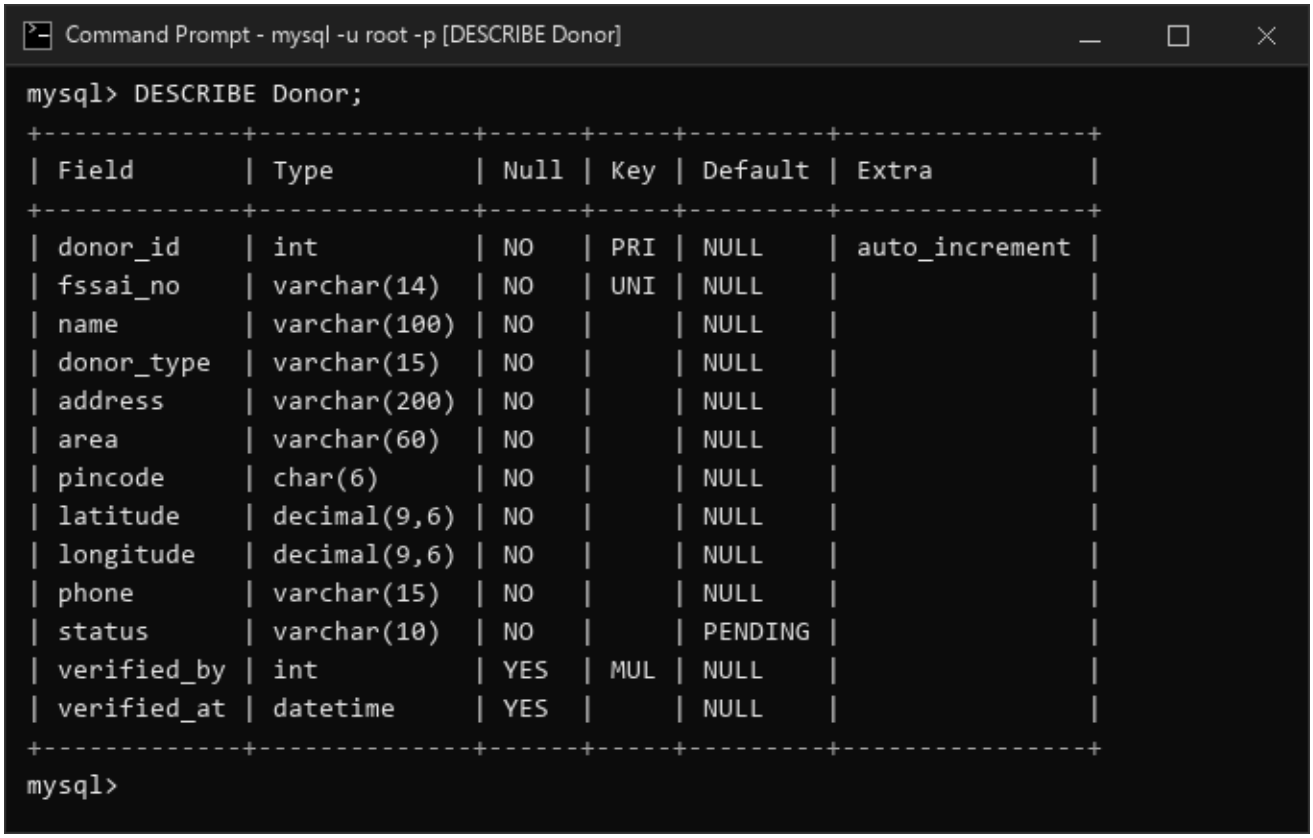


Figure: MySQL Terminal: DESCRIBE Donor showing PK, FK, and status defaults

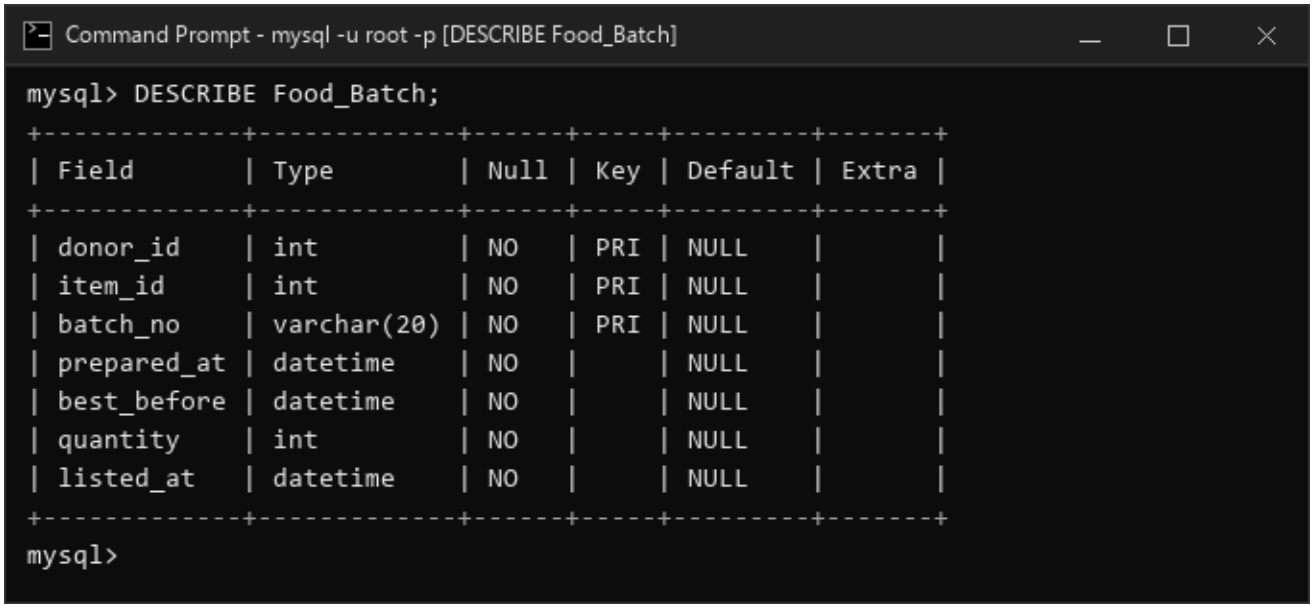


Figure: MySQL Terminal: DESCRIBE Food_Batch showing composite PK and timestamp fields

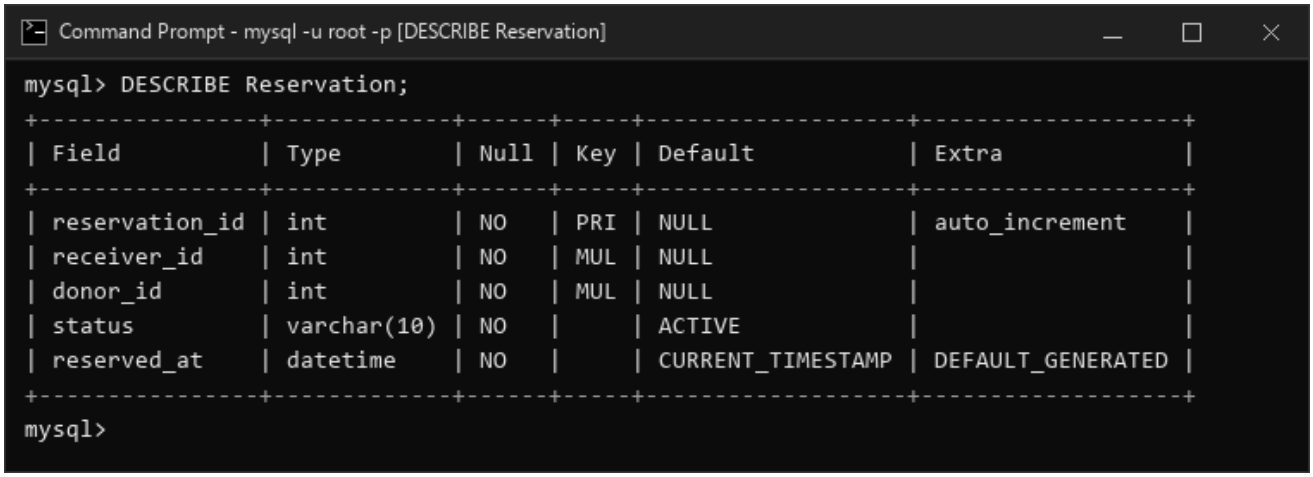


Figure: MySQL Terminal: DESCRIBE Reservation showing FKs and status constraints

5. Sample Data Population (DML) & Verification

Sample data was designed to cover comprehensive operational edge cases: (1) Multiple donor types (restaurants, bakeries, caterers, supermarkets); (2) Donor status variations (VERIFIED, PENDING verification, and REJECTED); (3) Real-world shelf-life distributions (batches expiring in 2 hours, today, and tomorrow); (4) Mixed reservation statuses (ACTIVE holds, COLLECTED, CANCELLED, and EXPIRED); (5) Zero-result search scenarios to test demand logging.

5.1 MySQL Terminal Screenshots of Sample Data Verification

Below are actual MySQL terminal execution screenshots confirming row population across all 15 tables:

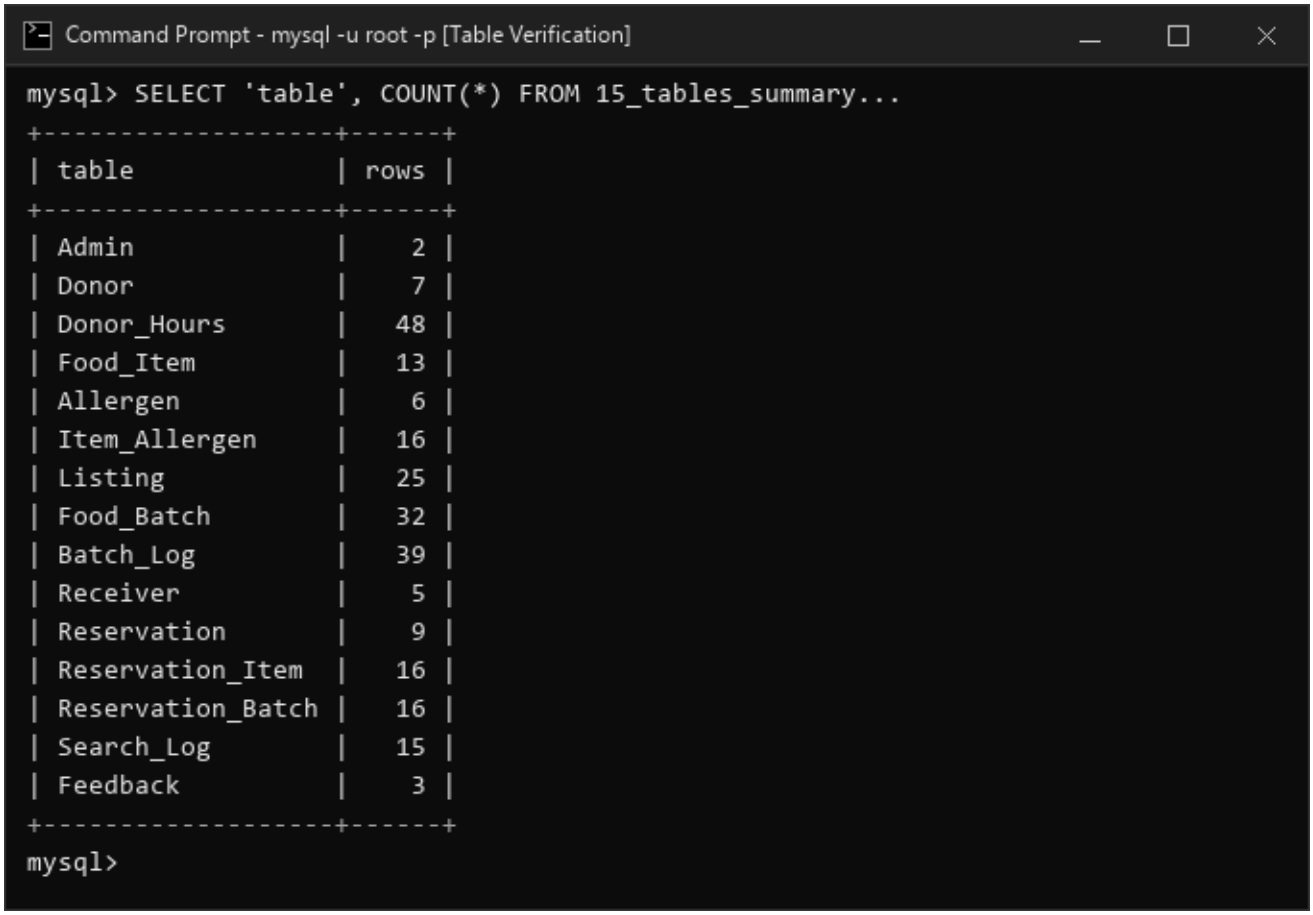


Figure: MySQL Terminal: Row count verification across all 15 normalized tables

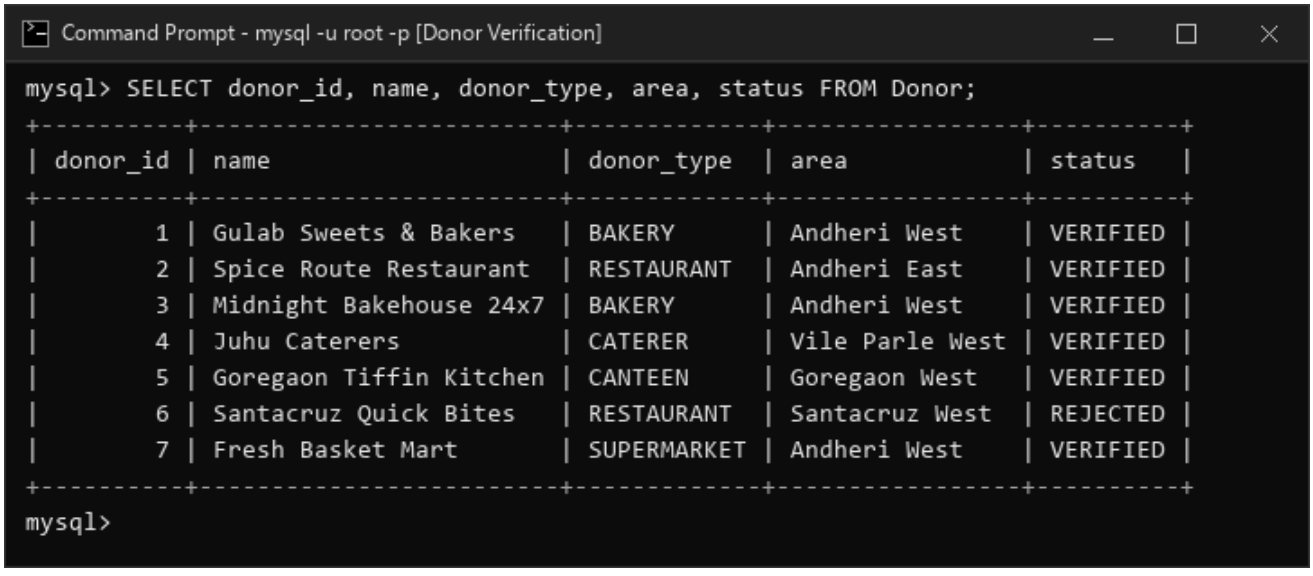


Figure: MySQL Terminal: Sample donor records showing status and classification distribution